

Plateforme analytique moderne sur Google Cloud

Astrafy — Take-Home Challenge (Data Engineer) · Partie 1 : Design Challenge

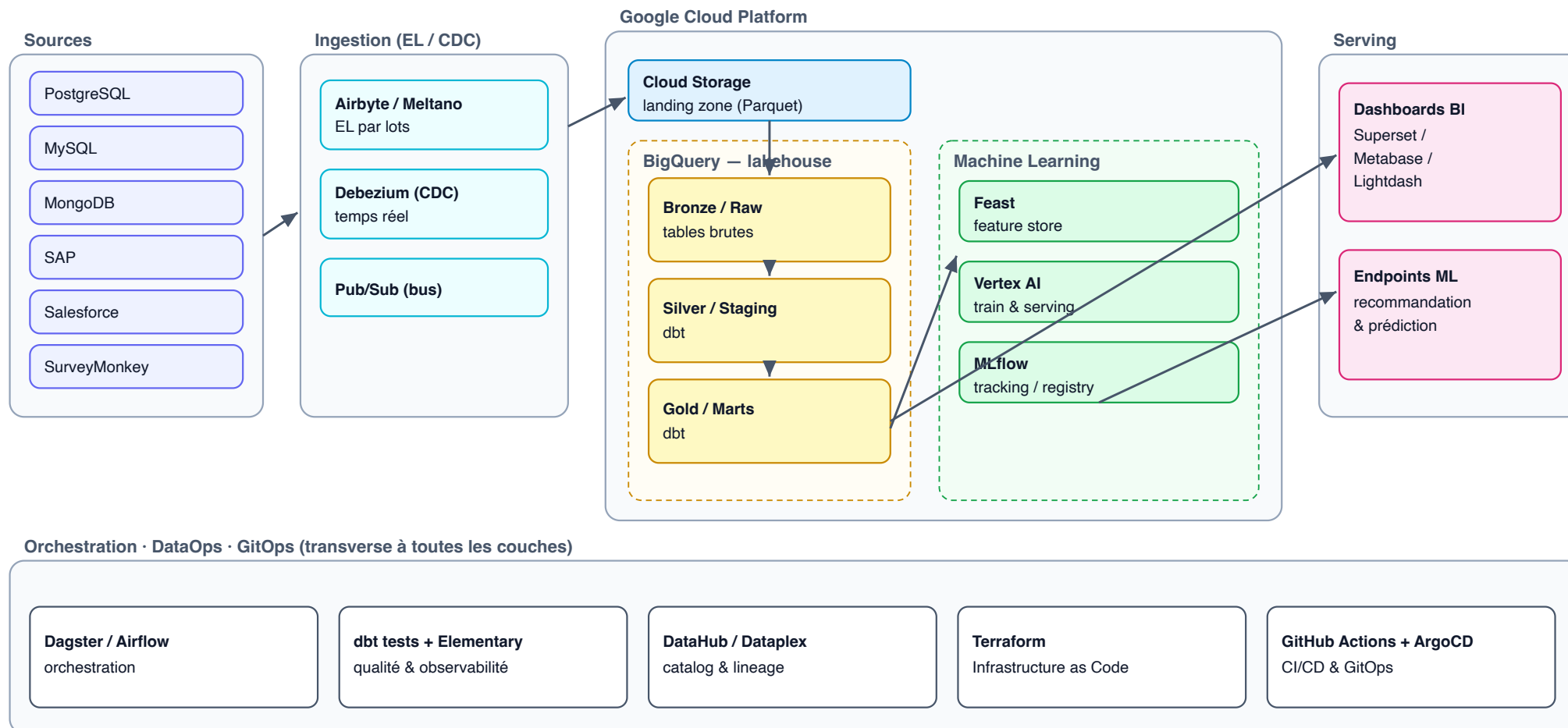
Contexte

Le client a plusieurs sources de données très différentes : des bases transactionnelles (PostgreSQL, MySQL, MongoDB) et des applications métier (SAP, Salesforce, SurveyMonkey). Il veut repartir sur une base analytique saine, capable à la fois d'alimenter les **dashboards BI** des différents départements et de servir de socle aux **modèles ML** de recommandation et de prédiction.

Exigences

- **Google Cloud** comme plateforme d'hébergement
- **Open-source** au maximum (limiter le lock-in)
- Fort accent sur le **GitOps** et le **DataOps**

Architecture cible — lakehouse GitOps



Lecture du flux : les données partent des sources, sont ingérées (par lots pour les applications, en CDC pour les bases transactionnelles), atterrissent sur Cloud Storage, puis remontent dans BigQuery en trois couches (Bronze, Silver, Gold) modélisées avec dbt. Elles sont ensuite consommées par la BI et par le ML. L'orchestration et la gouvernance restent transverse à toutes les couches.

Choix technologiques par couche

Couche	Choix (open-source privilégié)	Rôle	Alternative GCP native
Ingestion (EL)	Airbyte / Meltano	Extraction & chargement multi-connecteurs (PostgreSQL, MySQL, MongoDB, Salesforce, SurveyMonkey)	Datastream
CDC temps réel	Debezium + Pub/Sub	Capture des changements sur les bases transactionnelles sans surcharger la source	Datastream
Landing	Cloud Storage	Zone d'atterrissage brute, immuable et rejouable (Parquet/Avro)	—
Warehouse / Lakehouse	BigQuery (+ BigLake/Iceberg)	Moteur analytique serverless ; médaille Bronze/Silver/Gold	BigQuery
Transformation	dbt-core	Modélisation SQL, tests, docs, lineage, environnements dev/prod	Dataform
Orchestration	Dagster / Airflow (Cloud Composer)	Planification & dépendances des pipelines (assets)	Cloud Composer
Qualité data	dbt tests + Elementary / Great Expectations	Tests, contrats de données, observabilité	Dataplex DQ
Machine Learning	Feast + MLflow (+ Vertex AI)	Feature store, tracking/registry, entraînement & serving	Vertex AI
BI	Superset / Metabase / Lightdash	Dashboards par département ; Lightdash s'appuie sur la couche sémantique dbt	Looker
Catalog / gouvernance	DataHub / OpenMetadata	Catalogue, lineage, data mesh, gouvernance	Dataplex
IaC / GitOps	Terraform + GitHub Actions + ArgoCD/Flux	Provisioning cloud & déploiement déclaratif des outils sur GKE	—

Pourquoi cette approche

- Les trois couches (Bronze, Silver, Gold) séparent le brut, le nettoyé et l'exposé. On sait à tout moment où on en est, et on peut tester chaque étage.
- On fait de l'ELT et non de l'ETL : on charge d'abord la donnée dans BigQuery, puis on la transforme avec dbt. C'est plus simple à maintenir, versionné dans Git et rejouable.
- On mélange batch et temps réel selon le besoin réel : CDC pour les bases transactionnelles où la fraîcheur compte, ingestion par lots pour les applications.
- Chaque brique open-source peut être remplacée sans tout casser, ce qui limite la dépendance à un fournisseur.

Sécurité & gouvernance

- IAM au moindre privilège, service accounts dédiés par usage.
- **Workload Identity Federation** : CI/CD sans clé statique.
- VPC Service Controls + Secret Manager pour cloisonner et protéger les secrets.
- Environnements **dev / staging / prod** isolés (datasets et/ou projets séparés).

Focus GitOps & DataOps

GitOps

- **Git = source de vérité** pour l'infra ET les données.
- **Terraform** pour toutes les ressources cloud (projet, BigQuery, GKE, IAM, réseau).
- Outils open-source déployés sur **GKE** via **Helm + ArgoCD/Flux** (état déclaratif réconcilié en continu).
- Toute modification passe par une **Pull Request** revue et validée.

DataOps

- **CI dbt sur PR** : build en environnement éphémère + tests, avec **Slim CI** (state:modified+) pour ne recalculer que l'impacté.
- **Contrats de données** & tests bloquants pour éviter les régressions.
- **Observabilité data** (Elementary) + monitoring infra (Prometheus/Grafana ou Cloud Monitoring).
- **Lineage** automatique via dbt + DataHub, du champ source au dashboard.

Compromis assumés

Aucune architecture n'est parfaite. Voici les principaux arbitrages et la logique derrière chacun.

Sujet	Décision et justification
BigQuery malgré l'exigence open-source	BigQuery reste le moteur analytique de référence sur GCP : serverless, scalable et économique. On assume ce choix managé, mais on garde la main sur la portabilité grâce à une couche de transformation indépendante (dbt) et, si le client le souhaite, à des formats de tables ouverts (BigLake / Iceberg sur Cloud Storage).
Connecteur SAP	C'est le point le plus délicat. Les connecteurs SAP côté Airbyte sont parfois incomplets. Si c'est le cas, on prévoit une extraction dédiée (SAP SLT ou export Datasphere) plutôt que de forcer un connecteur inadapté.
Batch ou temps réel	On ne met pas du streaming partout. Le CDC est réservé aux sources où la fraîcheur est vraiment utile ; le reste passe par du batch, plus simple à exploiter.
Managé ou auto-hébergé	On auto-héberge les outils open-source sur GKE (Airbyte, Dagster, Superset, DataHub) pour garder la maîtrise et maîtriser les coûts. On bascule sur du managé (Vertex AI, Composer) quand l'effort d'exploitation ne se justifie pas.

Lien avec la Partie 2. Le code livré met en pratique cette vision à petite échelle : Terraform crée le projet GCP, les datasets BigQuery et le service account ; dbt matérialise une couche staging puis un data mart ; GitHub Actions lance dbt sur chaque pull request, avec une authentification par Workload Identity Federation (donc sans clé stockée).